GKE Vault Secret Management Guide

https://gemini.google.com/app/6544a055a300c2dc

User prompt: pavankalyanmahanthi@cloudshell:~ (gcpproject-502010)$ ll total 26224 drwxr-x--- 20 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 22 10:11 ./ drwxr-xr-x 4 root root 4096 Apr 25 04:28 ../ drwxrwxr-x 4 pavankalyanmahanthi pavankalyanmahanthi 4096 Jun 25 10:17 .ansible/ -rw------- 1 pavankalyanmahanthi pavankalyanmahanthi 23959 Jul 22 10:13 .bash_history -rw-r--r-- 1 pavankalyanmahanthi pavankalyanmahanthi 220 Mar 31 2024 .bash_logout -rw-r--r-- 1 pavankalyanmahanthi pavankalyanmahanthi 3868 Jun 25 10:12 .bashrc drwxrwxr-x 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Jun 25 09:03 bin/ drwx------ 7 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 8 09:02 .cache/ drwx------ 4 pavankalyanmahanthi pavankalyanmahanthi 4096 Apr 25 04:32 .codeoss/ drwxr-xr-x 5 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 8 09:02 .config/ -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi 836 Jul 22 10:11 deployment.yaml drwxrwxr-x 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Apr 25 04:28 .docker/ -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi 218 Jul 22 10:01 Dockerfile drwxrwxr-x 4 pavankalyanmahanthi pavankalyanmahanthi 4096 Jun 25 10:15 Gcp_Devops/ -rw------- 1 pavankalyanmahanthi pavankalyanmahanthi 0 Jun 23 09:40 gcp-sa-key.json drwxrwxr-x 5 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 22 09:36 .gemini/ -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi 75 Jul 10 14:20 .gitconfig drwxrwxr-x 5 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 10 16:10 gitops-sec-demo./ drwxrwxr-x 3 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 22 09:44 .kube/ drwxrwxr-x 4 pavankalyanmahanthi pavankalyanmahanthi 4096 Jun 25 10:11 .local/ drwxrwxr-x 3 pavankalyanmahanthi pavankalyanmahanthi 4096 Apr 25 04:32 .npm/ -rw-r--r-- 1 pavankalyanmahanthi pavankalyanmahanthi 807 Mar 31 2024 .profile -rwxr-xr-x 1 pavankalyanmahanthi pavankalyanmahanthi 913 Jul 22 09:36 README-cloudshell.txt -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi 12 Jul 22 10:01 requirements.txt drwxrwxr-x 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Jun 25 10:11 .rustup/ drwx------ 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Jun 25 10:25 .ssh/ -rw-r--r-- 1 pavankalyanmahanthi pavankalyanmahanthi 0 Apr 25 04:28 .sudo_as_admin_successful -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi 26690615 Apr 10 2024 terraform_1.8.0_linux_amd64.zip drwxr-xr-x 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Jun 25 09:04 .terraform.d/ drwxrwxr-x 3 pavankalyanmahanthi pavankalyanmahanthi 4096 Apr 26 12:47 test/ drwxrwxr-x 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 22 10:01 Vault_Practice/ -rw------- 1 pavankalyanmahanthi pavankalyanmahanthi 14675 Jul 10 16:27 .viminfo drwxr-xr-x 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Apr 25 04:32 .vscode/ -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi 228 Jun 25 09:03 .wget-hsts pavankalyanmahanthi@cloudshell:~ (gcpproject-502010)$ cd Vault_Practice/ pavankalyanmahanthi@cloudshell:~/Vault_Practice (gcpproject-502010)$ ll total 8 drwxrwxr-x 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 22 10:01 ./ drwxr-x--- 20 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 22 10:11 ../ pavankalyanmahanthi@cloudshell:~/Vault_Practice (gcpproject-502010)$ cd .. pavankalyanmahanthi@cloudshell:~ (gcpproject-502010)$ mkdir vault-app pavankalyanmahanthi@cloudshell:~ (gcpproject-502010)$ cd vault-app pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ mv ../app.py ../Dockerfile ../requirements.txt . mv: cannot stat '../app.py': No such file or directory*

Response: Looking at your `ll` output, `app.py` was missing from your home directory, which is why `mv` threw that error.

Here is the quick fix to create `app.py`, build the image cleanly, and get your pod running.

Copy and paste these commands into Cloud Shell:

Step 1: Create `app.py` in your `vault-app` directory

```
# Ensure you are inside ~/vault-app
cd ~/vault-app

# Move the Dockerfile and requirements.txt from home if you haven't already
mv ../Dockerfile ../requirements.txt . 2>/dev/null || true

# Write app.py
cat <<'EOF' > app.py
import time
import os
import logging
import sys
import hvac

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[logging.StreamHandler(sys.stdout)]
)

VAULT_ADDR = os.getenv("VAULT_ADDR", "http://vault.default.svc.cluster.local:8200")
K8S_TOKEN_PATH = "/var/run/secrets/kubernetes.io/serviceaccount/token"
VAULT_ROLE = "myapp-role"
SECRET_PATH = "myapp/config"

def get_sa_jwt_token():
    try:
        with open(K8S_TOKEN_PATH, "r") as f:
            return f.read().strip()
    except Exception as e:
        logging.error(f"Failed to read Service Account token: {e}")
        sys.exit(1)

def get_vault_client():
    jwt_token = get_sa_jwt_token()
    client = hvac.Client(url=VAULT_ADDR)
```

```
    try:
        client.auth.kubernetes.login(role=VAULT_ROLE, jwt=jwt_token)
        if client.is_authenticated():
            logging.info("Successfully authenticated with Vault via Kubernetes Auth.")
            return client
        else:
            logging.error("Vault authentication failed.")
            sys.exit(1)
    except Exception as e:
        logging.error(f"Error authenticating with Vault: {e}")
        sys.exit(1)

def main():
    logging.info("Starting Application and initializing Vault client...")
    client = get_vault_client()

    while True:
        try:
            secret_response = client.secrets.kv.v2.read_secret_version(
                path=SECRET_PATH,
                mount_point="secret"
            )
            password = secret_response['data']['data']['password']
            logging.info(f"FETCHED SECRET SUCCESSFULLY -> Password: {password}")
        except hvac.exceptions.Forbidden:
            logging.warning("Vault token expired or forbidden. Re-authenticating...")
            client = get_vault_client()
        except Exception as e:
            logging.error(f"Unexpected error fetching secret: {e}")

        time.sleep(10)

if __name__ == "__main__":
    main()
EOF
```

Step 2: Enable Cloud Build API & Submit Build

Now that all three files (`app.py`, `Dockerfile`, `requirements.txt`) are present inside `~/vault-app`, build and push the container image:

```
# Enable Cloud Build API
gcloud services enable cloudbuild.googleapis.com

# Set project & image environment variables
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1"
export FULL_IMAGE_PATH="${REGION}-docker.pkg.dev/${PROJECT_ID}/vault-demo-repo/vault-python-app:v1"

# Build and push via Cloud Build
gcloud builds submit --tag ${FULL_IMAGE_PATH} .
```

Step 3: Trigger Pod Restart & Check Logs

Once the build finishes successfully, delete the broken pod so Kubernetes pulls the newly available image:

```
# Delete the failing pod to force a restart
kubectl delete pod -l app=vault-python-app

# Stream logs from the new pod
kubectl logs -f -l app=vault-python-app
```

You should now see the log entries confirming authentication with Vault and fetching your secret password every 10 seconds!

User prompt: pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ ll total 20 drwxrwxr-x 2 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 22 10:16 ./ drwxr-x--- 21 pavankalyanmahanthi pavankalyanmahanthi 4096 Jul 22 10:15 ../ -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi 1972 Jul 22 10:16 app.py -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi 218 Jul 22 10:01 Dockerfile -rw-rw-r-- 1 pavankalyanmahanthi pavankalyanmahanthi   12 Jul 22 10:01 requirements.txt pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # Enable Cloud Build API gcloud services enable cloudbuild.googleapis.com # Set project & image environment variables export PROJECT_ID=$(gcloud config get-value project) export REGION="us-central1" export FULL_IMAGE_PATH="${REGION}-docker.pkg.dev/${PROJECT_ID}/vault-demo-repo/vault-python-app:v1" # Build and push via Cloud Build gcloud builds submit --tag ${FULL_IMAGE_PATH} . Operation "operations/acf.p2-186513045263-cbf213b4-9cf6-44da-a998-d675b8df7aa5" finished successfully. Your active configuration is: [cloudshell-25294] Creating temporary archive of 3 file(s) totalling 2.2 KiB before compression. Uploading tarball of [.] to [gs://gcpproject-502010_cloudbuild/source/1784715441.563918-d20c14d789f348ebafaa088c42b67184.tgz] ERROR: (gcloud.builds.submit)

PERMISSION_DENIED: The caller does not have permission. This command is authenticated as pavankalyanmahanthi@gmail.com which is the active account specified by the [core/account] property

Response: That `PERMISSION_DENIED` on Cloud Build happens when your active Google identity doesn't have the **Cloud Build Editor** role, or when the default Cloud Build Service Account lacks permissions to access Artifact Registry.

Since we are already running in Google Cloud Shell, **Docker is pre-installed locally**. We can completely bypass Cloud Build, build the container image directly using local Docker, and push it straight to Artifact Registry!

Run these commands in your `~/vault-app` directory:

Step 1: Configure Local Docker Authentication & Build Image

```
# Set environment variables
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1"
export FULL_IMAGE_PATH="${REGION}-docker.pkg.dev/${PROJECT_ID}/vault-demo-repo/vault-python-app:v1"

# Authenticate local Docker with Google Artifact Registry
gcloud auth configure-docker ${REGION}-docker.pkg.dev --quiet

# Build the Docker image locally
docker build -t ${FULL_IMAGE_PATH} .

# Push the image directly to Artifact Registry
docker push ${FULL_IMAGE_PATH}
```

Step 2: Restart the Pod and Stream Logs

Now that the image is officially uploaded to Artifact Registry, delete the stuck pod so Kubernetes pulls the new image and starts running:

```
# Delete the failing pod to force a fresh pull
kubectl delete pod -l app=vault-python-app

# Stream the logs
kubectl logs -f -l app=vault-python-app
```

Step 3: Test Dynamic Secret Rotation (Section 7 Verification)

While keeping your logs open in one terminal, open a **new terminal tab** in Cloud Shell and run:

```
# Update the password inside Vault
kubectl exec -it vault-0 -- vault kv put secret/myapp/config password="RotatedSuperSecret456!"
```

In your original log terminal, you will see the Python app automatically pick up `RotatedSuperSecret456!` within 10 seconds **without any pod restart**.

User prompt: pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # Set environment variables export PROJECT_ID=$(gcloud config get-value project) export REGION="us-central1" export FULL_IMAGE_PATH="${REGION}-docker.pkg.dev/${PROJECT_ID}/vault-demo-repo/vault-python-app:v1" # Authenticate local Docker with Google Artifact Registry gcloud auth configure-docker ${REGION}-docker.pkg.dev --quiet # Build the Docker image locally docker build -t ${FULL_IMAGE_PATH} . # Push the image directly to Artifact Registry docker push ${FULL_IMAGE_PATH} Your active configuration is: [cloudshell-25294] WARNING: Your config file at [/home/pavankalyanmahanthi/.docker/config.json] contains these credential helper entries: { "credHelpers": { "gcr.io": "gcloud", "us.gcr.io": "gcloud", "eu.gcr.io": "gcloud", "asia.gcr.io": "gcloud", "staging-k8s.gcr.io": "gcloud", "marketplace.gcr.io": "gcloud", "africa-south1-docker.pkg.dev": "gcloud", "asia-docker.pkg.dev": "gcloud", "asia-east1-docker.pkg.dev": "gcloud", "asia-east2-docker.pkg.dev": "gcloud", "asia-northeast1-docker.pkg.dev": "gcloud", "asia-northeast2-docker.pkg.dev": "gcloud", "asia-northeast3-docker.pkg.dev": "gcloud", "asia-south1-docker.pkg.dev": "gcloud", "asia-south2-docker.pkg.dev": "gcloud", "asia-southeast1-docker.pkg.dev": "gcloud", "asia-southeast2-docker.pkg.dev": "gcloud", "asia-southeast3-docker.pkg.dev": "gcloud", "australia-southeast1-docker.pkg.dev": "gcloud", "australia-southeast2-docker.pkg.dev": "gcloud", "docker.africa-south1.rep.pkg.dev": "gcloud", "docker.asia-east1.rep.pkg.dev": "gcloud", "docker.asia-east2.rep.pkg.dev": "gcloud", "docker.asia-northeast1.rep.pkg.dev": "gcloud", "docker.asia-northeast2.rep.pkg.dev": "gcloud", "docker.asia-northeast3.rep.pkg.dev": "gcloud", "docker.asia-south1.rep.pkg.dev": "gcloud", "docker.asia-south2.rep.pkg.dev": "gcloud", "docker.asia-southeast1.rep.pkg.dev": "gcloud", "docker.asia-southeast2.rep.pkg.dev": "gcloud", "docker.australia-southeast1.rep.pkg.dev": "gcloud", "docker.australia-southeast2.rep.pkg.dev": "gcloud", "docker.europe-central2.rep.pkg.dev": "gcloud", "docker.europe-north1.rep.pkg.dev": "gcloud", "docker.europe-southwest1.rep.pkg.dev": "gcloud", "docker.europe-west1.rep.pkg.dev": "gcloud", "docker.europe-west10.rep.pkg.dev": "gcloud", "docker.europe-west12.rep.pkg.dev": "gcloud", "docker.europe-west2.rep.pkg.dev": "gcloud", "docker.europe-west3.rep.pkg.dev": "gcloud", "docker.europe-west4.rep.pkg.dev": "gcloud", "docker.europe-west6.rep.pkg.dev": "gcloud", "docker.europe-west8.rep.pkg.dev": "gcloud", "docker.europe-west9.rep.pkg.dev": "gcloud", "docker.me-central1.rep.pkg.dev": "gcloud", "docker.me-central2.rep.pkg.dev": "gcloud", "docker.me-west1.rep.pkg.dev": "gcloud", "docker.northamerica-northeast1.rep.pkg.dev": "gcloud", "docker.northamerica-northeast2.rep.pkg.dev": "gcloud", "docker.southamerica-east1.rep.pkg.dev": "gcloud", "docker.southamerica-west1.rep.pkg.dev": "gcloud", "docker.us-central1.rep.pkg.dev": "gcloud", "docker.us-central2.rep.pkg.dev": "gcloud", "docker.us-east1.rep.pkg.dev": "gcloud", "docker.us-east4.rep.pkg.dev": "gcloud", "docker.us-east5.rep.pkg.dev": "gcloud", "docker.us-east7.rep.pkg.dev": "gcloud", "docker.us-south1.rep.pkg.dev": "gcloud", "docker.us-west1.rep.pkg.dev": "gcloud", "docker.us-west2.rep.pkg.dev": "gcloud", "docker.us-west3.rep.pkg.dev": "gcloud", "docker.us-west4.rep.pkg.dev": "gcloud", "europe-central2-docker.pkg.dev": "gcloud", "europe-docker.pkg.dev": "gcloud", "europe-north1-docker.pkg.dev": "gcloud", "europe-north2-docker.pkg.dev":

"gcloud", "europe-southwest1-docker.pkg.dev": "gcloud", "europe-west1-docker.pkg.dev": "gcloud", "europe-west10-docker.pkg.dev": "gcloud", "europe-west12-docker.pkg.dev": "gcloud", "europe-west2-docker.pkg.dev": "gcloud", "europe-west3-docker.pkg.dev": "gcloud", "europe-west4-docker.pkg.dev": "gcloud", "europe-west6-docker.pkg.dev": "gcloud", "europe-west8-docker.pkg.dev": "gcloud", "europe-west9-docker.pkg.dev": "gcloud", "me-central1-docker.pkg.dev": "gcloud", "me-central2-docker.pkg.dev": "gcloud", "me-west1-docker.pkg.dev": "gcloud", "northamerica-northeast1-docker.pkg.dev": "gcloud", "northamerica-northeast2-docker.pkg.dev": "gcloud", "northamerica-south1-docker.pkg.dev": "gcloud", "southamerica-east1-docker.pkg.dev": "gcloud", "southamerica-west1-docker.pkg.dev": "gcloud", "us-central1-docker.pkg.dev": "gcloud", "us-central2-docker.pkg.dev": "gcloud", "us-docker.pkg.dev": "gcloud", "us-east1-docker.pkg.dev": "gcloud", "us-east4-docker.pkg.dev": "gcloud", "us-east5-docker.pkg.dev": "gcloud", "us-east7-docker.pkg.dev": "gcloud", "us-south1-docker.pkg.dev": "gcloud", "us-west1-docker.pkg.dev": "gcloud", "us-west2-docker.pkg.dev": "gcloud", "us-west3-docker.pkg.dev": "gcloud", "us-west4-docker.pkg.dev": "gcloud", "us-west8-docker.pkg.dev": "gcloud" } } Adding credentials for: us-central1-docker.pkg.dev gcloud credential helpers already registered correctly. [+] Building 19.9s (10/10) FINISHED docker:default => [internal] load build definition from Dockerfile 0.0s => => transferring dockerfile: 257B 0.0s => [internal] load metadata for docker.io/library/python:3.11-slim 1.3s => [internal] load .dockerignore 0.0s => => transferring context: 2B 0.0s => [1/5] FROM docker.io/library/python:3.11-slim@sha256:db3ff2e1800a8581e2c48a27c3995339d47bdf046da21c7627accd3d51053a93 13.2s => => resolve docker.io/library/python:3.11-slim@sha256:db3ff2e1800a8581e2c48a27c3995339d47bdf046da21c7627accd3d51053a93 0.0s => => sha256:c89b9f64c028c19ba92e195d7589d914b9cd1fc69f3c3dfeb931e93876ac2064 248B / 248B 0.2s => => sha256:6b265b8eae4a26a263eb55546aa430c1eef37032a7cd86393d98c6b779f28319 14.42MB / 14.42MB 3.3s => => sha256:9775d166087aba0afe5ba9a88859c04c08090e82ae218742d09fec9119e0335d 1.29MB / 1.29MB 0.7s => => sha256:062e450697faa5f02a3a74eba9864ee4d79bc9cfbd65769fc6cdff2c05c6a053 29.78MB / 29.78MB 11.0s => => extracting sha256:062e450697faa5f02a3a74eba9864ee4d79bc9cfbd65769fc6cdff2c05c6a053 1.2s => => extracting sha256:9775d166087aba0afe5ba9a88859c04c08090e82ae218742d09fec9119e0335d 0.1s => => extracting sha256:6b265b8eae4a26a263eb55546aa430c1eef37032a7cd86393d98c6b779f28319 0.8s => => extracting sha256:c89b9f64c028c19ba92e195d7589d914b9cd1fc69f3c3dfeb931e93876ac2064 0.0s => [internal] load build context 0.0s => => transferring context: 2.06kB 0.0s => [2/5] WORKDIR /app 0.7s => [3/5] COPY requirements.txt . 0.0s => [4/5] RUN pip install --no-cache-dir -r requirements.txt 3.7s => [5/5] COPY app.py . 0.0s => exporting to image 0.7s => => exporting layers 0.4s => => exporting manifest sha256:9b3127ff7ef8cefe2854b7d09f7caa668c59cf9dd810d76bf9271974ce31aa86 0.0s => => exporting config sha256:3a9215c74a36538a20e6d6d2893c2c005c0cd18457e562800722abf59646c7cc 0.0s => => exporting attestation manifest sha256:302b4c6c8b1be372d35c75aa174b8776bf7ea7465b50ca03d4a82ff23d98ad6a 0.0s => => exporting manifest list sha256:1e096162633f5bf4653dfd57c2d8d45ed032da7ede515a20d48d3f14a48ef0a7 0.0s => => naming to us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 0.0s => => unpacking to us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 0.2s The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] c9ad4f30fa03: Waiting f91881e89963: Waiting 2757826b8b83: Waiting 062e450697fa: Waiting 9775d166087a: Waiting 6b265b8eae4a: Waiting 78c3609dab77: Waiting c89b9f64c028: Waiting 57edb8dca9dc: Waiting failed to do request: Head "https://us-central1-docker.pkg.dev/v2/gcpproject-502010/vault-demo-repo/vault-python-app/blobs/sha256:c9ad4f30fa0353761eb9383518526856a416c5a13084d6d712cab6e8b4ecda4d": dial tcp 142.251.107.82:443: connect: connection refused pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # Delete the failing pod to force a fresh pull kubectl delete pod -l app=vault-python-app # Stream the logs kubectl logs -f -l app=vault-python-app pod "vault-python-app-7d68cbc49f-54xzn" deleted from default namespace Error from server (BadRequest): container "app" in pod "vault-python-app-7d68cbc49f-wst8n" is waiting to start: ContainerCreating

Response: Your `docker push` failed because of a network connection issue when attempting to reach Google Artifact Registry (`dial tcp: connect: connection refused`). Because the image was never successfully uploaded, Kubernetes can't pull it, keeping your pod stuck in `ContainerCreating` or `ImagePullBackOff`.

How to Fix It

Step 1: Ensure the Artifact Registry Repository Exists & API is Enabled

Before pushing, ensure the `vault-demo-repo` repository actually exists in `us-central1` and the Artifact Registry API is active.

Run these commands in Cloud Shell:

```
# 1. Enable Artifact Registry API
gcloud services enable artifactregistry.googleapis.com

# 2. Check if the repository exists (or create it if missing)
gcloud artifacts repositories create vault-demo-repo \
    --repository-format=docker \
    --location=us-central1 \
    --description="Docker repository for Vault demo" \
    --quiet 2>/dev/null || echo "Repository already exists."
```

Step 2: Retry Authentication & Push

Refresh Docker's authentication helper for Artifact Registry and re-run the push:

```
# Re-authenticate Docker with Artifact Registry
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet

# Push the built image again
docker push ${FULL_IMAGE_PATH}
```

> **Note on `connection refused`:** If `docker push` still fails with `connection refused` after running the above, Cloud Shell may have had a temporary network hiccup or an active web proxy setting interfering with TCP traffic. Try clearing transient environment proxies with `unset http_proxy https_proxy` before retrying `docker push`.

Step 3: Trigger a Fresh Kubernetes Deployment

Once `docker push` finishes successfully with output showing layer digests:

```
# Restart the deployment so Kubernetes pulls the updated image
kubectl rollout restart deployment/vault-python-app

# Watch the pod status until it reaches Running
kubectl get pods -l app=vault-python-app -w
```

User prompt: pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # Re-authenticate Docker with Artifact Registry gcloud auth configure-docker us-central1-docker.pkg.dev --quiet # Push the built image again docker push ${FULL_IMAGE_PATH} WARNING: Your config file at [/home/pavankalyanmahanthi/.docker/config.json] contains these credential helper entries: { "credHelpers": { "gcr.io": "gcloud", "us.gcr.io": "gcloud", "eu.gcr.io": "gcloud", "asia.gcr.io": "gcloud", "staging-k8s.gcr.io": "gcloud", "marketplace.gcr.io": "gcloud", "africa-south1-docker.pkg.dev": "gcloud", "asia-docker.pkg.dev": "gcloud", "asia-east1-docker.pkg.dev": "gcloud", "asia-east2-docker.pkg.dev": "gcloud", "asia-northeast1-docker.pkg.dev": "gcloud", "asia-northeast2-docker.pkg.dev": "gcloud", "asia-northeast3-docker.pkg.dev": "gcloud", "asia-south1-docker.pkg.dev": "gcloud", "asia-south2-docker.pkg.dev": "gcloud", "asia-southeast1-docker.pkg.dev": "gcloud", "asia-southeast2-docker.pkg.dev": "gcloud", "asia-southeast3-docker.pkg.dev": "gcloud", "australia-southeast1-docker.pkg.dev": "gcloud", "australia-southeast2-docker.pkg.dev": "gcloud", "docker.africa-south1.rep.pkg.dev": "gcloud", "docker.asia-east1.rep.pkg.dev": "gcloud", "docker.asia-east2.rep.pkg.dev": "gcloud", "docker.asia-northeast1.rep.pkg.dev": "gcloud", "docker.asia-northeast2.rep.pkg.dev": "gcloud", "docker.asia-northeast3.rep.pkg.dev": "gcloud", "docker.asia-south1.rep.pkg.dev": "gcloud", "docker.asia-south2.rep.pkg.dev": "gcloud", "docker.asia-southeast1.rep.pkg.dev": "gcloud", "docker.asia-southeast2.rep.pkg.dev": "gcloud", "docker.australia-southeast1.rep.pkg.dev": "gcloud", "docker.australia-southeast2.rep.pkg.dev": "gcloud", "docker.europe-central2.rep.pkg.dev": "gcloud", "docker.europe-north1.rep.pkg.dev": "gcloud", "docker.europe-southwest1.rep.pkg.dev": "gcloud", "docker.europe-west1.rep.pkg.dev": "gcloud", "docker.europe-west10.rep.pkg.dev": "gcloud", "docker.europe-west12.rep.pkg.dev": "gcloud", "docker.europe-west2.rep.pkg.dev": "gcloud", "docker.europe-west3.rep.pkg.dev": "gcloud", "docker.europe-west4.rep.pkg.dev": "gcloud", "docker.europe-west6.rep.pkg.dev": "gcloud", "docker.europe-west8.rep.pkg.dev": "gcloud", "docker.europe-west9.rep.pkg.dev": "gcloud", "docker.me-central1.rep.pkg.dev": "gcloud", "docker.me-central2.rep.pkg.dev": "gcloud", "docker.me-west1.rep.pkg.dev": "gcloud", "docker.northamerica-northeast1.rep.pkg.dev": "gcloud", "docker.northamerica-northeast2.rep.pkg.dev": "gcloud", "docker.southamerica-east1.rep.pkg.dev": "gcloud", "docker.southamerica-west1.rep.pkg.dev": "gcloud", "docker.us-central1.rep.pkg.dev": "gcloud", "docker.us-central2.rep.pkg.dev": "gcloud", "docker.us-east1.rep.pkg.dev": "gcloud", "docker.us-east4.rep.pkg.dev": "gcloud", "docker.us-east5.rep.pkg.dev": "gcloud", "docker.us-east7.rep.pkg.dev": "gcloud", "docker.us-south1.rep.pkg.dev": "gcloud", "docker.us-west1.rep.pkg.dev": "gcloud", "docker.us-west2.rep.pkg.dev": "gcloud", "docker.us-west3.rep.pkg.dev": "gcloud", "docker.us-west4.rep.pkg.dev": "gcloud", "europe-central2-docker.pkg.dev": "gcloud", "europe-docker.pkg.dev": "gcloud", "europe-north1-docker.pkg.dev": "gcloud", "europe-north2-docker.pkg.dev": "gcloud", "europe-southwest1-docker.pkg.dev": "gcloud", "europe-west1-docker.pkg.dev": "gcloud", "europe-west10-docker.pkg.dev": "gcloud", "europe-west12-docker.pkg.dev": "gcloud", "europe-west2-docker.pkg.dev": "gcloud", "europe-west3-docker.pkg.dev": "gcloud", "europe-west4-docker.pkg.dev": "gcloud", "europe-west6-docker.pkg.dev": "gcloud", "europe-west8-docker.pkg.dev": "gcloud", "europe-west9-docker.pkg.dev": "gcloud", "me-central1-docker.pkg.dev": "gcloud", "me-central2-docker.pkg.dev": "gcloud", "me-west1-docker.pkg.dev": "gcloud", "northamerica-northeast1-docker.pkg.dev": "gcloud", "northamerica-northeast2-docker.pkg.dev": "gcloud", "northamerica-south1-docker.pkg.dev": "gcloud", "southamerica-east1-docker.pkg.dev": "gcloud", "southamerica-west1-docker.pkg.dev": "gcloud", "us-central1-docker.pkg.dev": "gcloud", "us-central2-docker.pkg.dev": "gcloud", "us-docker.pkg.dev": "gcloud", "us-east1-docker.pkg.dev": "gcloud", "us-east4-docker.pkg.dev": "gcloud", "us-east5-docker.pkg.dev": "gcloud", "us-east7-docker.pkg.dev": "gcloud", "us-south1-docker.pkg.dev": "gcloud", "us-west1-docker.pkg.dev": "gcloud", "us-west2-docker.pkg.dev": "gcloud", "us-west3-docker.pkg.dev": "gcloud", "us-west4-docker.pkg.dev": "gcloud", "us-west8-docker.pkg.dev": "gcloud" } } Adding credentials for: us-central1-docker.pkg.dev gcloud credential helpers already registered correctly. The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 6b265b8eae4a: Waiting c89b9f64c028: Waiting 2757826b8b83: Waiting c9ad4f30fa03: Waiting f91881e89963: Waiting 78c3609dab77: Waiting 57edb8dca9dc: Waiting 062e450697fa: Waiting 9775d166087a: Waiting failed to do request: Head "https://us-central1-docker.pkg.dev/v2/gcpproject-502010/vault-demo-repo/vault-python-app/blobs/sha256:2757826b8b835ba360a96e33ae016dac8606ce1e5de3da67614710a94639f996": dial tcp 172.253.147.82:443: connect: connection refused

Response: The error `dial tcp ...:443: connect: connection refused` occurs when the **Docker daemon** inside Cloud Shell cannot establish an outbound HTTPS connection to Google Artifact Registry.

This is usually caused by **stale network proxy settings** or **Docker daemon session glitches** in Cloud Shell.

Solutions (Try in Order)

Option 1: Unset Proxies & Restart Docker Service (Recommended)

Cloud Shell sometimes sets environment proxy variables that prevent `dockerd` from reaching GCP endpoints.

Run the following commands in Cloud Shell to clear proxy variables and restart Docker:

```
# 1. Unset environment proxy variables
unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY

# 2. Restart the Docker daemon in Cloud Shell
sudo systemctl restart docker

# 3. Re-authenticate with Artifact Registry
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet

# 4. Retry the push
docker push ${FULL_IMAGE_PATH}
```

Option 2: Push via Access Token Login

If `gcloud auth configure-docker` credential helper continues to fail, log Docker directly into the registry using an explicit OAuth access token:

```
# Explicitly log Docker into us-central1-docker.pkg.dev
gcloud auth print-access-token | docker login -u oauth2accesstoken --password-stdin https://us-central1-docker.pkg.dev

# Retry pushing the image
docker push ${FULL_IMAGE_PATH}
```

Option 3: Verify Variable Value & Repository Connectivity

Ensure `${FULL_IMAGE_PATH}` is defined correctly and that Cloud Shell can connect to Artifact Registry's endpoint:

```
# Check if FULL_IMAGE_PATH is set correctly
echo "Image path is: ${FULL_IMAGE_PATH}"

# Test HTTPS connection directly
curl -v https://us-central1-docker.pkg.dev/v2/
```

> **Expected output from curl:** A `401 Unauthorized` response means the network connection is working properly and ready for authentication.

Next Steps Once Pushed

Once the push succeeds, restart your Kubernetes deployment:

```
kubectl rollout restart deployment/vault-python-app
kubectl get pods -l app=vault-python-app -w
```

User prompt: pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # 1. Unset environment proxy variables unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY # 2. Restart the Docker daemon in Cloud Shell sudo systemctl restart docker # 3. Re-authenticate with Artifact Registry gcloud auth configure-docker us-central1-docker.pkg.dev --quiet # 4. Retry the push docker push ${FULL_IMAGE_PATH} System has not been booted with systemd as init system (PID 1). Can't operate. Failed to connect to bus: Host is down WARNING: Your config file at [/home/pavankalyanmahanthi/.docker/config.json] contains these credential helper entries: { "credHelpers": { "gcr.io": "gcloud", "us.gcr.io": "gcloud", "eu.gcr.io": "gcloud", "asia.gcr.io": "gcloud", "staging-k8s.gcr.io": "gcloud", "marketplace.gcr.io": "gcloud", "africa-south1-docker.pkg.dev": "gcloud", "asia-docker.pkg.dev": "gcloud", "asia-east1-docker.pkg.dev": "gcloud", "asia-east2-docker.pkg.dev": "gcloud", "asia-northeast1-docker.pkg.dev": "gcloud", "asia-northeast2-docker.pkg.dev": "gcloud", "asia-northeast3-docker.pkg.dev": "gcloud", "asia-south1-docker.pkg.dev": "gcloud", "asia-south2-docker.pkg.dev": "gcloud", "asia-southeast1-docker.pkg.dev": "gcloud", "asia-southeast2-docker.pkg.dev": "gcloud", "asia-southeast3-docker.pkg.dev": "gcloud", "australia-southeast1-docker.pkg.dev": "gcloud", "australia-southeast2-docker.pkg.dev": "gcloud", "docker.africa-south1.rep.pkg.dev": "gcloud", "docker.asia-east1.rep.pkg.dev": "gcloud", "docker.asia-east2.rep.pkg.dev": "gcloud", "docker.asia-northeast1.rep.pkg.dev": "gcloud", "docker.asia-northeast2.rep.pkg.dev": "gcloud", "docker.asia-northeast3.rep.pkg.dev": "gcloud", "docker.asia-south1.rep.pkg.dev": "gcloud", "docker.asia-south2.rep.pkg.dev": "gcloud", "docker.asia-southeast1.rep.pkg.dev": "gcloud", "docker.asia-southeast2.rep.pkg.dev": "gcloud", "docker.australia-southeast1.rep.pkg.dev": "gcloud", "docker.australia-southeast2.rep.pkg.dev": "gcloud", "docker.europe-central2.rep.pkg.dev": "gcloud", "docker.europe-north1.rep.pkg.dev": "gcloud", "docker.europe-southwest1.rep.pkg.dev": "gcloud", "docker.europe-west1.rep.pkg.dev": "gcloud", "docker.europe-west10.rep.pkg.dev": "gcloud", "docker.europe-west12.rep.pkg.dev": "gcloud", "docker.europe-west2.rep.pkg.dev": "gcloud", "docker.europe-west3.rep.pkg.dev": "gcloud", "docker.europe-west4.rep.pkg.dev": "gcloud", "docker.europe-west6.rep.pkg.dev": "gcloud", "docker.europe-west8.rep.pkg.dev": "gcloud", "docker.europe-west9.rep.pkg.dev": "gcloud", "docker.me-central1.rep.pkg.dev": "gcloud", "docker.me-central2.rep.pkg.dev": "gcloud", "docker.me-west1.rep.pkg.dev": "gcloud", "docker.northamerica-northeast1.rep.pkg.dev": "gcloud", "docker.northamerica-northeast2.rep.pkg.dev": "gcloud", "docker.southamerica-east1.rep.pkg.dev": "gcloud", "docker.southamerica-west1.rep.pkg.dev": "gcloud", "docker.us-central1.rep.pkg.dev": "gcloud", "docker.us-central2.rep.pkg.dev": "gcloud", "docker.us-east1.rep.pkg.dev": "gcloud", "docker.us-east4.rep.pkg.dev": "gcloud", "docker.us-east5.rep.pkg.dev": "gcloud", "docker.us-east7.rep.pkg.dev": "gcloud", "docker.us-south1.rep.pkg.dev": "gcloud", "docker.us-west1.rep.pkg.dev": "gcloud", "docker.us-west2.rep.pkg.dev": "gcloud", "docker.us-west3.rep.pkg.dev": "gcloud", "docker.us-west4.rep.pkg.dev": "gcloud", "europe-central2-docker.pkg.dev": "gcloud", "europe-docker.pkg.dev": "gcloud", "europe-north1-docker.pkg.dev": "gcloud", "europe-north2-docker.pkg.dev": "gcloud", "europe-southwest1-docker.pkg.dev": "gcloud", "europe-west1-docker.pkg.dev": "gcloud", "europe-west10-docker.pkg.dev": "gcloud", "europe-west12-docker.pkg.dev": "gcloud", "europe-west2-docker.pkg.dev": "gcloud", "europe-west3-docker.pkg.dev": "gcloud",

"europe-west4-docker.pkg.dev": "gcloud", "europe-west6-docker.pkg.dev": "gcloud", "europe-west8-docker.pkg.dev": "gcloud", "europe-west9-docker.pkg.dev": "gcloud", "me-central1-docker.pkg.dev": "gcloud", "me-central2-docker.pkg.dev": "gcloud", "me-west1-docker.pkg.dev": "gcloud", "northamerica-northeast1-docker.pkg.dev": "gcloud", "northamerica-northeast2-docker.pkg.dev": "gcloud", "northamerica-south1-docker.pkg.dev": "gcloud", "southamerica-east1-docker.pkg.dev": "gcloud", "southamerica-west1-docker.pkg.dev": "gcloud", "us-central1-docker.pkg.dev": "gcloud", "us-central2-docker.pkg.dev": "gcloud", "us-docker.pkg.dev": "gcloud", "us-east1-docker.pkg.dev": "gcloud", "us-east4-docker.pkg.dev": "gcloud", "us-east5-docker.pkg.dev": "gcloud", "us-east7-docker.pkg.dev": "gcloud", "us-south1-docker.pkg.dev": "gcloud", "us-west1-docker.pkg.dev": "gcloud", "us-west2-docker.pkg.dev": "gcloud", "us-west3-docker.pkg.dev": "gcloud", "us-west4-docker.pkg.dev": "gcloud", "us-west8-docker.pkg.dev": "gcloud" } } Adding credentials for: us-central1-docker.pkg.dev gcloud credential helpers already registered correctly. The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] c9ad4f30fa03: Waiting f91881e89963: Waiting 6b265b8eae4a: Waiting c89b9f64c028: Waiting 062e450697fa: Waiting 9775d166087a: Waiting 2757826b8b83: Waiting 78c3609dab77: Waiting 57edb8dca9dc: Waiting failed to do request: Head "https://us-central1-docker.pkg.dev/v2/gcpproject-502010/vault-demo-repo/vault-python-app/blobs/sha256:2757826b8b835ba360a96e33ae016dac8606ce1e5de3da67614710a94639f996": dial tcp 142.251.107.82:443: connect: connection refused

Response: Cloud Shell runs inside a containerized environment (which is why `systemctl` failed—it doesn't use `systemd`). When Docker within Cloud Shell throws `connection refused` on `142.251.107.82:443`, the underlying Docker daemon is blocked by stale environment variables or a hung daemon process.

Here are the exact fixes to resolve this in Cloud Shell:

Step 1: Restart Docker using SysV init (works in Cloud Shell)

Since `systemctl` doesn't work, use the `service` command to restart Docker:

```
sudo service docker restart
```

Step 2: Clear Docker Daemon Proxy Overrides

Cloud Shell often injects proxy settings directly into the Docker daemon configuration. Clear them out and test connectivity:

```
# 1. Clear terminal proxy variables for current session
unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY

# 2. Check if Docker daemon has proxy settings in its system config
sudo grep -rn "proxy" /etc/default/docker /etc/systemd/system/ /etc/docker/ 2>/dev/null
```

If any files show `HTTP_PROXY` or `HTTPS_PROXY` settings, open them with `sudo nano <filepath>` and comment those lines out with `#`, then run `sudo service docker restart`.

Step 3: Test Connectivity & Retry Push

```
# 1. Verify Cloud Shell can hit Artifact Registry
curl -v https://us-central1-docker.pkg.dev/v2/
# Expected result: HTTP/2 401 (This means the network path is open)

# 2. Re-authenticate
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet

# 3. Push the image
docker push ${FULL_IMAGE_PATH}
```

Step 4: The Ultimate Fallback (Restart Cloud Shell VM)

If the Docker daemon remains hung:

1. Click the **3 dots (⋮)** in the top right corner of the Cloud Shell window.
2. Click **Restart**.
3. Once reconnected, navigate back and run:

   ```
   cd ~/vault-app
   gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
   docker push ${FULL_IMAGE_PATH}
   ```

Step 5: Resume Kubernetes Deployment

Once the push succeeds:

```
kubectl rollout restart deployment/vault-python-app
kubectl get pods -l app=vault-python-app -w
```

---

*User prompt: pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ sudo service docker restart * Stopping Docker: docker [OK] * Starting Docker: docker [OK] pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # 1. Clear terminal proxy variables for current session unset http_proxy https_proxy HTTP_PROXY HTTPS_PROXY # 2. Check if Docker daemon has proxy settings in its system config sudo grep -rn "proxy" /etc/default/docker /etc/systemd/system/ /etc/docker/ 2>/dev/null /etc/default/docker:16:# If you need Docker to use an HTTP proxy, it can also be specified here. /etc/default/docker:17:#export http_proxy="http://127.0.0.1:3128/" pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # 1. Verify Cloud Shell can hit Artifact Registry curl -v https://us-central1-docker.pkg.dev/v2/ # Expected result: HTTP/2 401 (This means the network path is open) # 2. Re-authenticate gcloud auth configure-docker us-central1-docker.pkg.dev --quiet # 3. Push the image docker push ${FULL_IMAGE_PATH} * Host us-central1-docker.pkg.dev:443 was resolved. * IPv6: 2607:f8b0:400c:c12::52 * IPv4: 142.250.98.82 * Trying 142.250.98.82:443... * Connected to us-central1-docker.pkg.dev (142.250.98.82) port 443 * ALPN: curl offers h2,http/1.1 * TLSv1.3 (OUT), TLS handshake, Client hello (1): * CAfile: /etc/ssl/certs/ca-certificates.crt * CApath: /etc/ssl/certs * TLSv1.3 (IN), TLS handshake, Server hello (2): * TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8): * TLSv1.3 (IN), TLS handshake, Certificate (11): * TLSv1.3 (IN), TLS handshake, CERT verify (15): * TLSv1.3 (IN), TLS handshake, Finished (20): * TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1): * TLSv1.3 (OUT), TLS handshake, Finished (20): * SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519 / id-ecPublicKey * ALPN: server accepted h2 * Server certificate: * subject: CN=*.googlecode.com * start date: Jun 29 08:39:20 2026 GMT * expire date: Sep 21 08:39:19 2026 GMT * subjectAltName: host "us-central1-docker.pkg.dev" matched cert's "*.pkg.dev" * issuer: C=US; O=Google Trust Services; CN=WE2 * SSL certificate verify ok. * Certificate level 0: Public key type EC/prime256v1 (256/128 Bits/secBits), signed using ecdsa-with-SHA256 * Certificate level 1: Public key type EC/prime256v1 (256/128 Bits/secBits), signed using ecdsa-with-SHA384 * Certificate level 2: Public key type EC/secp384r1 (384/192 Bits/secBits), signed using ecdsa-with-SHA384 * using HTTP/2 * [HTTP/2] [1] OPENED stream for https://us-central1-docker.pkg.dev/v2/ * [HTTP/2] [1] [:method: GET] * [HTTP/2] [1] [:scheme: https] * [HTTP/2] [1] [:authority: us-central1-docker.pkg.dev] * [HTTP/2] [1] [:path: /v2/] * [HTTP/2] [1] [user-agent: curl/8.5.0] * [HTTP/2] [1] [accept: */*] > GET /v2/ HTTP/2 > Host: us-central1-docker.pkg.dev > User-Agent: curl/8.5.0 > Accept: */* > < HTTP/2 401 < content-type: application/json; charset=utf-8 < docker-distribution-api-version: registry/2.0 < www-authenticate: Bearer realm="https://us-central1-docker.pkg.dev/v2/token" < x-content-type-options: nosniff < x-frame-options: SAMEORIGIN < x-xss-protection: 0 < date: Wed, 22 Jul 2026 10:24:34 GMT < content-length: 96 < {"errors":[{"code":"UNAUTHORIZED","message":"not authenticated: No credential was supplied."}]} * Connection #0 to host us-central1-docker.pkg.dev left intact WARNING: Your config file at [/home/pavankalyanmahanthi/.docker/config.json] contains these credential helper entries: { "credHelpers": { "gcr.io": "gcloud", "us.gcr.io": "gcloud", "eu.gcr.io": "gcloud", "asia.gcr.io": "gcloud", "staging-k8s.gcr.io": "gcloud", "marketplace.gcr.io": "gcloud", "africa-south1-docker.pkg.dev": "gcloud", "asia-docker.pkg.dev": "gcloud", "asia-east1-docker.pkg.dev": "gcloud", "asia-east2-docker.pkg.dev": "gcloud", "asia-northeast1-docker.pkg.dev": "gcloud", "asia-northeast2-docker.pkg.dev": "gcloud", "asia-northeast3-docker.pkg.dev": "gcloud", "asia-south1-docker.pkg.dev": "gcloud", "asia-south2-docker.pkg.dev": "gcloud", "asia-southeast1-docker.pkg.dev": "gcloud", "asia-southeast2-docker.pkg.dev": "gcloud", "asia-southeast3-docker.pkg.dev": "gcloud", "australia-southeast1-docker.pkg.dev": "gcloud", "australia-southeast2-docker.pkg.dev": "gcloud", "docker.africa-south1.rep.pkg.dev": "gcloud", "docker.asia-east1.rep.pkg.dev": "gcloud", "docker.asia-east2.rep.pkg.dev": "gcloud", "docker.asia-northeast1.rep.pkg.dev": "gcloud", "docker.asia-northeast2.rep.pkg.dev": "gcloud", "docker.asia-northeast3.rep.pkg.dev": "gcloud", "docker.asia-south1.rep.pkg.dev": "gcloud", "docker.asia-south2.rep.pkg.dev": "gcloud", "docker.asia-southeast1.rep.pkg.dev": "gcloud", "docker.asia-southeast2.rep.pkg.dev": "gcloud", "docker.australia-southeast1.rep.pkg.dev": "gcloud", "docker.australia-southeast2.rep.pkg.dev": "gcloud", "docker.europe-central2.rep.pkg.dev": "gcloud", "docker.europe-north1.rep.pkg.dev": "gcloud", "docker.europe-southwest1.rep.pkg.dev": "gcloud", "docker.europe-west1.rep.pkg.dev": "gcloud", "docker.europe-west10.rep.pkg.dev": "gcloud", "docker.europe-west12.rep.pkg.dev": "gcloud", "docker.europe-west2.rep.pkg.dev": "gcloud", "docker.europe-west3.rep.pkg.dev": "gcloud", "docker.europe-west4.rep.pkg.dev": "gcloud", "docker.europe-west6.rep.pkg.dev": "gcloud", "docker.europe-west8.rep.pkg.dev": "gcloud", "docker.europe-west9.rep.pkg.dev": "gcloud", "docker.me-central1.rep.pkg.dev": "gcloud", "docker.me-central2.rep.pkg.dev": "gcloud", "docker.me-west1.rep.pkg.dev": "gcloud", "docker.northamerica-northeast1.rep.pkg.dev": "gcloud", "docker.northamerica-northeast2.rep.pkg.dev": "gcloud", "docker.southamerica-east1.rep.pkg.dev": "gcloud", "docker.southamerica-west1.rep.pkg.dev": "gcloud", "docker.us-central1.rep.pkg.dev": "gcloud", "docker.us-central2.rep.pkg.dev": "gcloud", "docker.us-east1.rep.pkg.dev": "gcloud", "docker.us-east4.rep.pkg.dev": "gcloud", "docker.us-east5.rep.pkg.dev": "gcloud", "docker.us-east7.rep.pkg.dev": "gcloud", "docker.us-south1.rep.pkg.dev": "gcloud", "docker.us-west1.rep.pkg.dev": "gcloud", "docker.us-west2.rep.pkg.dev": "gcloud", "docker.us-west3.rep.pkg.dev": "gcloud", "docker.us-west4.rep.pkg.dev": "gcloud", "europe-central2-docker.pkg.dev": "gcloud", "europe-docker.pkg.dev": "gcloud", "europe-north1-docker.pkg.dev": "gcloud", "europe-north2-docker.pkg.dev": "gcloud", "europe-southwest1-docker.pkg.dev": "gcloud", "europe-west1-docker.pkg.dev": "gcloud", "europe-west10-docker.pkg.dev": "gcloud", "europe-west12-docker.pkg.dev": "gcloud", "europe-west2-docker.pkg.dev": "gcloud", "europe-west3-docker.pkg.dev": "gcloud", "europe-west4-docker.pkg.dev": "gcloud", "europe-west6-docker.pkg.dev": "gcloud", "europe-west8-docker.pkg.dev": "gcloud", "europe-west9-docker.pkg.dev": "gcloud", "me-central1-docker.pkg.dev": "gcloud", "me-central2-docker.pkg.dev": "gcloud", "me-west1-docker.pkg.dev": "gcloud", "northamerica-northeast1-docker.pkg.dev": "gcloud", "northamerica-northeast2-docker.pkg.dev": "gcloud", "northamerica-south1-docker.pkg.dev": "gcloud", "southamerica-east1-docker.pkg.dev": "gcloud", "southamerica-west1-docker.pkg.dev": "gcloud", "us-central1-docker.pkg.dev": "gcloud", "us-central2-docker.pkg.dev": "gcloud", "us-docker.pkg.dev": "gcloud", "us-east1-docker.pkg.dev": "gcloud", "us-east4-docker.pkg.dev": "gcloud", "us-east5-docker.pkg.dev": "gcloud", "us-east7-docker.pkg.dev": "gcloud", "us-south1-docker.pkg.dev": "gcloud", "us-west1-docker.pkg.dev": "gcloud", "us-west2-docker.pkg.dev": "gcloud", "us-west3-docker.pkg.dev": "gcloud", "us-west4-docker.pkg.dev": "gcloud", "us-west8-docker.pkg.dev": "gcloud" } } Adding credentials for: us-central1-docker.pkg.dev gcloud credential helpers already registered correctly. The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 57edb8dca9dc: Waiting 2757826b8b83: Waiting f91881e89963: Waiting 78c3609dab77: Waiting 6b265b8eae4a: Waiting c89b9f64c028: Waiting 062e450697fa: Waiting 9775d166087a: Waiting c9ad4f30fa03: Waiting failed to do request: Head "https://us-central1-docker.pkg.dev/v2/gcpproject-502010/vault-demo-repo/vault-python-app/blobs/sha256:57edb8dca9dce7660b0ca5afbe76403efbf08160d4cfb63d6a0ee0d2e1cbc3cb": dial tcp 142.250.98.82:443: connect: connection refused pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$*

Response: The `curl` test returned **`HTTP/2 401`**, which confirms that your Cloud Shell terminal network is completely open and reaching Google Artifact Registry fine.

However, when `docker push` runs, the **Docker daemon background process** (which has its own isolated environment) is still trying to route traffic through a broken local proxy or bad socket connection.

Since Docker daemon restarts didn't clean its internal runtime state, here are two immediate ways to bypass or fix this.

Option 1: Direct Authentication Bypass (Fastest Fix)

Log Docker directly into Artifact Registry using an explicit OAuth token rather than relying on the `gcloud` credential helper:

```
# 1. Log in directly using your gcloud access token
gcloud auth print-access-token | docker login -u oauth2accesstoken --password-stdin https://us-central1-docker.pkg.dev

# 2. Push the image
docker push ${FULL_IMAGE_PATH}
```

Option 2: Clear Docker Daemon Proxy Override Files

Cloud Shell creates default proxy configurations inside `/etc/systemd/system/docker.service.d/` or `/etc/default/docker`. Even if unset in your shell, Docker reads those configuration files when spawned.

Run this script to locate and neutralize any proxy entries in Docker's runtime configuration:

```
# 1. Check if Docker systemd/service environment files exist with proxy settings
sudo grep -i "proxy" /etc/systemd/system/docker.service.d/*.conf 2>/dev/null

# 2. If any proxy file exists in systemd config, back it up and clear it
sudo rm -f /etc/systemd/system/docker.service.d/*proxy*.conf

# 3. Flush settings and restart Docker daemon
sudo service docker restart

# 4. Try pushing again
docker push ${FULL_IMAGE_PATH}
```

Option 3: Reset Cloud Shell Environment (If Option 1 & 2 fail)

If the Docker daemon remains stuck in a corrupted network state:

1. In the upper right corner of the Cloud Shell window, click the **3 dots (⋮)** menu.
2. Select **Restart**.
3. Once back online, re-run:

   ```
   cd ~/vault-app
   gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
   docker push ${FULL_IMAGE_PATH}
   ```

Next Step After Push Succeeds

Once you see the image layers finish uploading:

```
kubectl rollout restart deployment/vault-python-app
kubectl get pods -l app=vault-python-app -w
```

User prompt: pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # 1. Log in directly using your gcloud access token gcloud auth print-access-token | docker login -u oauth2accesstoken --password-stdin https://us-central1-docker.pkg.dev # 2. Push the image docker push ${FULL_IMAGE_PATH} Login Succeeded The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 062e450697fa: Waiting 9775d166087a: Waiting 78c3609dab77: Waiting 6b265b8eae4a: Waiting c89b9f64c028: Waiting c9ad4f30fa03: Waiting f91881e89963: Waiting 57edb8dca9dc: Waiting 2757826b8b83: Waiting failed to do request: Head "https://us-central1-docker.pkg.dev/v2/gcpproject-502010/vault-demo-repo/vault-python-app/blobs/sha256:78c3609dab77f4494a526761c349f5ba54a3155b491958a6627cd587937a9f17": dial tcp 142.251.107.82:443: connect: connection refused

Response: Even with direct authentication working, the underlying Docker daemon in this specific Cloud Shell instance is stuck attempting to route blob upload requests through an invalid or dead local network socket.

Because Cloud Shell ephemeral instances can sometimes lock up their background `dockerd` network stack, **restarting the Cloud Shell VM** is the fastest and cleanest way to clear the bad state.

Step 1: Restart Cloud Shell

1. Look at the top right corner of your Cloud Shell window.
2. Click the **three vertical dots (⋮)** menu.
3. Select **Restart**.

(Don't worry—your files in `~/vault-app` will be preserved.)

Step 2: Push Image & Roll Out Deployment

Once Cloud Shell reopens, run this sequence to push the image and update Kubernetes:

```
cd ~/vault-app

# 1. Verify environment variable is set
export FULL_IMAGE_PATH="us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1"
```

```
# 2. Re-authenticate Docker
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet

# 3. Push the image
docker push ${FULL_IMAGE_PATH}
```

Step 3: Verify Pod Health

Once the push completes and you see layer digests uploaded:

```
# Restart the deployment to pick up the image
kubectl rollout restart deployment/vault-python-app

# Watch the pod status transition to Running
kubectl get pods -l app=vault-python-app -w
```

User prompt: Adding credentials for: us-central1-docker.pkg.dev gcloud credential helpers already registered correctly. The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] f91881e89963: Waiting 57edb8dca9dc: Waiting c89b9f64c028: Waiting 2757826b8b83: Waiting 9775d166087a: Waiting c9ad4f30fa03: Waiting 78c3609dab77: Waiting 6b265b8eae4a: Waiting 062e450697fa: Waiting failed to do request: Head "https://us-central1-docker.pkg.dev/v2/gcpproject-502010/vault-demo-repo/vault-python-app/blobs/sha256:57edb8dca9dce7660b0ca5afbe76403efbf08160d4cfb63d6a0ee0d2e1cbc3cb": dial tcp 142.250.98.82:443: connect: connection refused

Response: Even after a shell restart, the Docker daemon inside this Cloud Shell instance is still failing outgoing TCP connections (`connection refused` on port 443). This happens when Cloud Shell's local Docker daemon gets stuck trying to route layer uploads through internal network proxy rules that are broken or blocked.

Instead of fighting the local Docker daemon, you can bypass local Docker completely by using **Google Cloud Build**. Cloud Build uploads your source code and builds/pushes the image directly inside Google's infrastructure.

The Solution: Use Cloud Build (Bypasses Local Docker)

Run this single command from your `~/vault-app` directory:

```
gcloud builds submit --tag us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1
```

Why this works:

- It reads your local `Dockerfile` in `~/vault-app`.
- It packages the source code and sends it to GCP's managed build service over standard HTTPS (which your `curl` test proved is open and working).
- Google builds and pushes the image directly into `vault-demo-repo` without touching your local Cloud Shell Docker daemon.

Next Steps: Deploy to Kubernetes

Once `gcloud builds submit` finishes with `STATUS: SUCCESS`:

```
# 1. Trigger a rollout restart so GKE fetches the new image
kubectl rollout restart deployment/vault-python-app

# 2. Watch your pod status turn to Running
kubectl get pods -l app=vault-python-app -w
```

User prompt: pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ gcloud builds submit --tag us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 Creating temporary archive of 3 file(s) totalling 2.2 KiB before compression. Uploading tarball of [.] to [gs://gcpproject-502010_cloudbuild/source/1784716461.911825-76bf8e81d9864475ba65ed52c358283b.tgz] ERROR: (gcloud.builds.submit) INVALID_ARGUMENT: could not resolve source: googleapi: Error 403: 186513045263-compute@developer.gserviceaccount.com does not have storage.objects.get access to the Google Cloud Storage object. Permission 'storage.objects.get' denied on resource '//storage.googleapis.com/projects/_/buckets/gcpproject-502010_cloudbuild/objects/source/1784716461.911825-76bf8e81d9864475ba65ed52c358283b.tgz' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2NC0wYzFhLTczY2YtYmNkNi02ZTVjYzMwOTk0YmYSdXByb2plY3RzL18vYnVja2V0cy9nY3., forbidden pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # 1. Trigger a rollout restart so GKE fetches the new image kubectl rollout restart deployment/vault-python-app # 2. Watch your pod status turn to Running kubectl get pods -l app=vault-python-app -w deployment.apps/vault-python-app restarted NAME READY STATUS RESTARTS AGE vault-python-app-665db4f56f-7bnqr 0/1 Pending 0 0s vault-python-app-7d68cbc49f-wst8n 0/1 ImagePullBackOff 0 15m vault-python-app-665db4f56f-7bnqr 0/1 ContainerCreating 0 0s vault-python-app-665db4f56f-7bnqr 0/1 ErrImagePull 0 2s vault-python-app-7d68cbc49f-wst8n 0/1 ErrImagePull 0 16m vault-python-app-665db4f56f-7bnqr 0/1 ImagePullBackOff 0 16s vault-python-app-

7d68cbc49f-wst8n 0/1 ImagePullBackOff 0 16m vault-python-app-665db4f56f-7bnqr 0/1 ErrImagePull 0 32s ^Cpavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$

Response: That Cloud Build error actually gives us the exact missing link!

The default Compute Engine service account (`186513045263-compute@developer.gserviceaccount.com`) lacks access to read Cloud Build's temporary storage bucket. Granting it the **Storage Object Viewer** role will let `gcloud builds submit` work seamlessly.

Once Cloud Build builds and pushes your container, the GKE pod will be able to pull it.

Fix Cloud Build Permissions & Build Image

Run these commands in Cloud Shell:

```
# 1. Get your project number
PROJECT_NUMBER=$(gcloud projects describe gcpproject-502010 --format="value(projectNumber)")

# 2. Grant Storage Object Viewer to the default Compute service account
gcloud projects add-iam-policy-binding gcpproject-502010 \
    --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \
    --role="roles/storage.objectViewer"

# 3. Submit the build again
gcloud builds submit --tag us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1
```

Deploy and Verify

Once you see `STATUS: SUCCESS` from Cloud Build:

```
# 1. Restart deployment so GKE fetches the newly pushed image
kubectl rollout restart deployment/vault-python-app

# 2. Watch the pod transition to Running
kubectl get pods -l app=vault-python-app -w
```

User prompt: pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ # 1. Get your project number PROJECT_NUMBER=$(gcloud projects describe gcpproject-502010 --format="value(projectNumber)") # 2. Grant Storage Object Viewer to the default Compute service account gcloud projects add-iam-policy-binding gcpproject-502010 \ --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \ --role="roles/storage.objectViewer" # 3. Submit the build again gcloud builds submit --tag us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 Updated IAM policy for project [gcpproject-502010]. bindings: - members: - serviceAccount:service-186513045263@gcp-sa-artifactregistry.iam.gserviceaccount.com role: roles/artifactregistry.serviceAgent - members: - serviceAccount:186513045263@cloudbuild.gserviceaccount.com role: roles/cloudbuild.builds.builder - members: - serviceAccount:service-186513045263@gcp-sa-cloudbuild.iam.gserviceaccount.com role: roles/cloudbuild.serviceAgent - members: - serviceAccount:186513045263@cloudservices.gserviceaccount.com role: roles/compute.instanceGroupManagerServiceAgent - members: - serviceAccount:service-186513045263@compute-system.iam.gserviceaccount.com role: roles/compute.serviceAgent - members: - serviceAccount:service-186513045263@gcp-sa-gkenode.iam.gserviceaccount.com role: roles/container.defaultNodeServiceAgent - members: - serviceAccount:service-186513045263@container-engine-robot.iam.gserviceaccount.com role: roles/container.serviceAgent - members: - serviceAccount:service-186513045263@containerregistry.iam.gserviceaccount.com role: roles/containerregistry.ServiceAgent - members: - serviceAccount:service-186513045263@cloud-filer.iam.gserviceaccount.com role: roles/file.serviceAgent - members: - serviceAccount:service-186513045263@gcp-sa-networkconnectivity.iam.gserviceaccount.com role: roles/networkconnectivity.serviceAgent - members: - user:pavankalyanmahanthi@gmail.com role: roles/owner - members: - serviceAccount:service-186513045263@gcp-sa-pubsub.iam.gserviceaccount.com role: roles/pubsub.serviceAgent - members: - serviceAccount:186513045263-compute@developer.gserviceaccount.com role: roles/storage.objectViewer etag: BwZXMLQnxzg= version: 1 Creating temporary archive of 3 file(s) totalling 2.2 KiB before compression. Uploading tarball of [.] to [gs://gcpproject-502010_cloudbuild/source/1784716553.874118-cb3acf6cedae443a97ca65907e3f04c3.tgz] Created [https://cloudbuild.googleapis.com/v1/projects/gcpproject-502010/locations/global/builds/b3f17853-4fb7-46a4-bce7-2905ef098774]. Logs are available at [https://console.cloud.google.com/cloud-build/builds/b3f17853-4fb7-46a4-bce7-2905ef098774?project=186513045263]. Waiting for build to complete. Polling interval: 1 second(s). ---------- ---------- REMOTE BUILD OUTPUT ---------- ---------- starting build "b3f17853-4fb7-46a4-bce7-2905ef098774" FETCHSOURCE Fetching storage object: gs://gcpproject-502010_cloudbuild/source/1784716553.874118-cb3acf6cedae443a97ca65907e3f04c3.tgz#1784716554111977 Copying gs://gcpproject-502010_cloudbuild/source/1784716553.874118-cb3acf6cedae443a97ca65907e3f04c3.tgz#1784716554111977... / [1 files][1.3 KiB/ 1.3 KiB] Operation completed over 1 objects/1.3 KiB. BUILD Already have image (with digest): gcr.io/cloud-builders/gcb-internal Sending build context to Docker daemon 5.632kB Step 1/9 : FROM python:3.11-slim 3.11-slim: Pulling from library/python 062e450697fa: Pulling fs layer 9775d166087a: Pulling fs layer 6b265b8eae4a: Pulling fs layer c89b9f64c028: Pulling fs layer c89b9f64c028: Verifying Checksum c89b9f64c028: Download complete 9775d166087a: Verifying Checksum 9775d166087a: Download complete 6b265b8eae4a: Verifying Checksum 6b265b8eae4a: Download complete 062e450697fa: Verifying Checksum 062e450697fa: Download complete 062e450697fa: Pull complete 9775d166087a: Pull complete 6b265b8eae4a: Pull complete c89b9f64c028: Pull complete Digest: sha256:db3ff2e1800a8581e2c48a27c3995339d47bdf046da21c7627accd3d51053a93 Status: Downloaded newer image for python:3.11-slim 1c03896dbf0e Step 2/9 : WORKDIR /app Running in 4eda9d8290ad Removing intermediate container 4eda9d8290ad 0f023d96321f Step 3/9 : ENV PYTHONDONTWRITEBYTECODE=1 Running in c48851d051be Removing intermediate container c48851d051be cb6814e354b5 Step 4/9 : ENV PYTHONUNBUFFERED=1 Running in 1be8fe46a9e8 Removing intermediate container 1be8fe46a9e8 68f4fa96434d Step 5/9 : COPY requirements.txt . 207d5a7e7a6d Step 6/9 : RUN pip install --no-cache-dir -r requirements.txt Running in 16aadf5f2026 Collecting hvac==2.3.0 (from -r requirements.txt (line 1)) Downloading hvac-2.3.0-py3-none-any.whl.metadata (3.3 kB) Collecting requests<3.0.0,>=2.27.1 (from hvac==2.3.0->-r requirements.txt (line 1)) Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB) Collecting charset_normalizer<4,>=2

(from requests<3.0.0,>=2.27.1->hvac==2.3.0->-r requirements.txt (line 1)) Downloading charset_normalizer-3.4.9-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (41 kB) ━━ 41.7/41.7 kB 28.7 MB/s eta 0:00:00 Collecting idna<4,>=2.5 (from requests<3.0.0,>=2.27.1->hvac==2.3.0->-r requirements.txt (line 1)) Downloading idna-3.18-py3-none-any.whl.metadata (6.1 kB) Collecting urllib3<3,>=1.26 (from requests<3.0.0,>=2.27.1->hvac==2.3.0->-r requirements.txt (line 1)) Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB) Collecting certifi>=2023.5.7 (from requests<3.0.0,>=2.27.1->hvac==2.3.0->-r requirements.txt (line 1)) Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB) Downloading hvac-2.3.0-py3-none-any.whl (155 kB) ━━ 155.9/155.9 kB 191.4 MB/s eta 0:00:00 Downloading requests-2.34.2-py3-none-any.whl (73 kB) ━━ 73.1/73.1 kB 233.2 MB/s eta 0:00:00 Downloading certifi-2026.7.22-py3-none-any.whl (136 kB) ━━ 137.0/137.0 kB 227.1 MB/s eta 0:00:00 Downloading charset_normalizer-3.4.9-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (221 kB) ━━ 221.3/221.3 kB 284.1 MB/s eta 0:00:00 Downloading idna-3.18-py3-none-any.whl (65 kB) ━━ 65.5/65.5 kB 200.9 MB/s eta 0:00:00 Downloading urllib3-2.7.0-py3-none-any.whl (131 kB) ━━ 131.1/131.1 kB 255.5 MB/s eta 0:00:00 Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests, hvac Successfully installed certifi-2026.7.22 charset_normalizer-3.4.9 hvac-2.3.0 idna-3.18 requests-2.34.2 urllib3-2.7.0 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv [notice] A new release of pip is available: 24.0 -> 26.1.2 [notice] To update, run: pip install --upgrade pip Removing intermediate container 16aadf5f2026 e331e9583c3a Step 7/9 : COPY app.py . 105da1292087 Step 8/9 : USER 10001 Running in b4a30859d600 Removing intermediate container b4a30859d600 58ca3019010e Step 9/9 : CMD ["python", "app.py"] Running in c63222f2225b Removing intermediate container c63222f2225b 25a0d8134b86 Successfully built 25a0d8134b86 Successfully tagged us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 PUSH Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2NS1kMzIzLTc2OWItOTljZS04ZGZkYzA3MGM4NjYSTXByb2plY3RzL2djcHByb2plY3QtNTAy . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2NS1INGIxLTc5YTgtYTgyMy0wMzc5YWQ3Y2E1OWISTXByb2plY3RzL2djcHByb2plY3QtNTA . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2NS1mYTJmLTc3YTQtYjE5My1iZDE1MzUyOGYwMDUSTXByb2plY3RzL2djcHByb2plY3QtN . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2Ni0wMTljLTdjZTQtODJmMC0yNDc1MGVlN2I4YTcSTXByb2plY3RzL2djcHByb2plY3QtNTAy . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2Ni0wYWZlLTcwZmItYmNiYi0yNjI5N2UwNWQ4OGUSTXByb2plY3RzL2djcHByb2plY3QtNTA . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2Ni0yMzMyLTc1ZmUtOTAxNy1iZTdjZTcyOWY3ZTgSTXByb2plY3RzL2djcHByb2plY3QtNTAy . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2Ni0yZjRjLTc4YWMtOTQxOS00NjgxZjA1Mjc0MDkSTXByb2plY3RzL2djcHByb2plY3QtNTAyM . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied:

Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2Ni0zY2ZiLTc5MmItOWFlOS02MWI5NDZjMTJjNjgSTXByb2plY3RzL2djcHByb2plY3QtNTAyM . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2Ni00ZDYyLTc0MTUtOGFhMi01YjljZmZhZmEzYTQSTXByb2plY3RzL2djcHByb2plY3QtNTAy . Pushing us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1 The push refers to repository [us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app] 4c9a500b7494: Preparing 572e69658022: Preparing 33223af1f986: Preparing 055519625986: Preparing fc95983f78e7: Preparing 99ab40bce873: Preparing 1d1e59fb3586: Preparing f2ec4de84f55: Preparing denied: Permission 'artifactregistry.repositories.uploadArtifacts' denied on resource '//artifactregistry.googleapis.com/projects/gcpproject-502010/locations/us-central1/repositories/vault-demo-repo' (or it may not exist). Remediate access with this Troubleshooter URL or share it with your administrator - https://console.cloud.google.com/iam-admin/troubleshooter/summary;errorId=CiQwMTlmODk2Ni02MjFhLTc2YTItYmFmNS1iYjQ5ZDMyZTUxYTMSTXByb2plY3RzL2djcHByb2plY3QtNTAy . ERROR: failed to push because we ran out of retries. ERROR ERROR: error pushing image "us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1": retry budget exhausted (10 attempts): step exited with non-zero status: 1 ---------- INFO: The service account running this build projects/gcpproject-502010/serviceAccounts/186513045263-compute@developer.gserviceaccount.com does not have permission to write logs to Cloud Logging. To fix this, grant the Logs Writer (roles/logging.logWriter) role to the service account. 1 message(s) issued. ERROR: (gcloud.builds.submit) build b3f17853-4fb7-46a4-bce7-2905ef098774 completed with status "FAILURE" pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$

Response: We're almost at the finish line! Cloud Build successfully retrieved your source files and compiled the container image.

The build failed right at the end because the Cloud Build worker (`186513045263-compute@developer.gserviceaccount.com`) lacks write permissions to Artifact Registry.

Granting it **Artifact Registry Writer** will allow it to push the built image directly to `vault-demo-repo`.

Step 1: Grant Artifact Registry Writer Role

Run this command to allow the Compute service account to write images into Artifact Registry:

```
gcloud projects add-iam-policy-binding gcpproject-502010 \
    --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \
    --role="roles/artifactregistry.writer"
```

Step 2: Resubmit Cloud Build

Now re-run the build submission:

```
gcloud builds submit --tag us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/vault-python-app:v1
```

(You should see it compile and conclude with `STATUS: SUCCESS`.)

Step 3: Trigger Rollout in GKE

Once the build finishes successfully, trigger GKE to fetch the new image:

```
# 1. Restart deployment
kubectl rollout restart deployment/vault-python-app

# 2. Watch your pods transition from ImagePullBackOff to Running
kubectl get pods -l app=vault-python-app -w
```

User prompt: above steps were good so far pavankalyanmahanthi@cloudshell:~/vault-app (gcpproject-502010)$ kubectl get pods -l app=vault-python-app -w NAME READY STATUS RESTARTS AGE vault-python-app-7bfbd94978-dxhq8 1/1 Running 0 87s

Response: Awesome! Seeing **`1/1 Running`** is the ultimate sign of victory here. Your Python application image built cleanly, pushed to Artifact Registry, and GKE pulled and spun it up without a hitch.

Now that the pod is up and running, here are a few quick checks you can run to verify that your application is talking to HashiCorp Vault properly:

1. Check Application Logs

Confirm that the app successfully authenticated to Vault and fetched its secrets:

```
kubectl logs -l app=vault-python-app --tail=50
```

What to look for: You should see startup logs indicating a successful connection or authentication response from your Vault server.

2. Test Vault Secret Retrieval (Exec into Pod)

If your app exposes an endpoint or environment variable sourced from Vault, you can exec into the container to inspect it:

```
# Get the pod name
POD_NAME=$(kubectl get pods -l app=vault-python-app -o jsonpath='{.items[0].metadata.name}')

# Check environment variables or test local endpoint
kubectl exec -it $POD_NAME -- env | grep -i vault
```

3. Verify GKE Service Output (If exposed)

If you have a Service routing traffic to `vault-python-app`:

```
kubectl get svc -l app=vault-python-app
```

User prompt: nice i can see logs , i can see conf values of vault pod i cant able to verify svc as nothing runs in it but thats okay how cai able to view this in python logs ?

Response: Awesome progress! Seeing those Vault configuration values in your pod means your app is successfully authenticating and loading parameters from Vault.

Since your service isn't serving external web traffic, viewing the exact output from Python comes down to how your `app.py` writes to standard stdout/stderr, combined with standard `kubectl` log tools.

Here are three ways to view and inspect those Python logs:

1. Streaming Live Logs in Real-Time

If your Python script runs in a loop or updates periodically, you can follow (tail) the log output live as it prints:

```
kubectl logs -f -l app=vault-python-app
```

Tip: Press `Ctrl + C` anytime to exit the live log stream.

2. Formatting Logs with Timestamp Context

To see exact timestamps for every `print()` or `logging` statement from your Python script:

```
kubectl logs -l app=vault-python-app --timestamps --tail=100
```

3. Python Logging Best Practices (Ensuring Output Appears)

If your logs appear blank or delayed, Python usually buffers `print()` outputs inside Docker containers by default.

In your `Dockerfile`, you already set `ENV PYTHONUNBUFFERED=1` (which disables output buffering), but in your `app.py`, make sure you are using standard logging so messages stream straight to `kubectl logs`:

```
import logging
import sys

# Configure stdout logging for Kubernetes
logging.basicConfig(
    stream=sys.stdout,
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s'
)

logging.info("Successfully fetched secrets from Vault!")
```

User prompt: pavankalyanmahanthi@cloudshell:~ (gcpproject-502010)$ kubectl logs -f -l app=vault-python-app 2026-07-22 12:03:12,253 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:03:22,258 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:03:32,263 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:03:42,269 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:03:52,275 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:04:02,281 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:04:12,287 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:04:22,293 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:04:32,298 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:04:42,305 [INFO] FETCHED SECRET SUCCESSFULLY

-> Password: InitialSuperSecret123! 2026-07-22 12:04:52,310 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:05:02,317 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:05:12,322 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123!

Response: That is a beauty to see! Every 10 seconds, your Python pod is successfully authenticating with HashiCorp Vault, reading the KV path, and extracting the updated secret without crashing or timing out.

That completes the end-to-end integration: **GKE Pod → Kubernetes Service Account → Vault Kubernetes Auth → Secret Retrieval**.

If you want to test how resilient your setup is, here are two cool tests you can run to see it in action dynamically:

Test 1: Update the Secret in Vault (Watch Python pick it up dynamically)

Exec into your Vault pod and update the password. Since your Python code polls every 10 seconds, you'll see the log stream instantly shift to printing the new password!

```
# 1. Exec into Vault pod
kubectl exec -it vault-0 -- vault kv put secret/myapp/config password="NewRotatedPassword2026!"

# 2. Watch your python logs update automatically on the next cycle
kubectl logs -f -l app=vault-python-app
```

Test 2: View Logs directly in GCP Cloud Logging (Console)

Because your Python app writes directly to `stdout`, GCP's Operations suite (formerly Stackdriver) automatically ingests these logs.

You can view them in Google Cloud Console or via `gcloud`:

```
gcloud logging read "resource.type=k8s_container AND resource.labels.pod_name:$POD_NAME" \
  --limit 10 \
  --format="value(textPayload)"
```

User prompt: pavankalyanmahanthi@cloudshell:~ (gcpproject-502010)$ # 1. Exec into Vault podod kubectl exec -it vault-0 -- vault kv put secret/myapp/config password="NewRotatedPassword2026!" # 2. Watch your python logs update automatically on the next cycle kubectl logs -f -l app=vault-python-app ====== Secret Path ====== secret/data/myapp/config ======= Metadata ======= Key Value --- ----- created_time 2026-07-22T12:05:54.622664237Z custom_metadata <nil> deletion_time n/a destroyed false version 2 2026-07-22 12:04:22,293 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:04:32,298 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:04:42,305 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:04:52,310 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:05:02,317 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:05:12,322 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:05:22,328 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:05:32,333 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:05:42,340 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:05:52,346 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123! 2026-07-22 12:06:02,352 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: NewRotatedPassword2026! 2026-07-22 12:06:12,357 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: NewRotatedPassword2026!

Response: That is awesome! Look right at the shift between timestamp `12:05:52` and `12:06:02`:

```
2026-07-22 12:05:52,346 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: InitialSuperSecret123!
2026-07-22 12:06:02,352 [INFO] FETCHED SECRET SUCCESSFULLY -> Password: NewRotatedPassword2026!
```

You updated Vault at `12:05:54` (`version 2`), and on the very next 10-second polling cycle, your Python app dynamically retrieved `NewRotatedPassword2026!` without needing a pod restart or deployment redeployment.

That completes a full, production-grade end-to-end Vault integration on GKE!

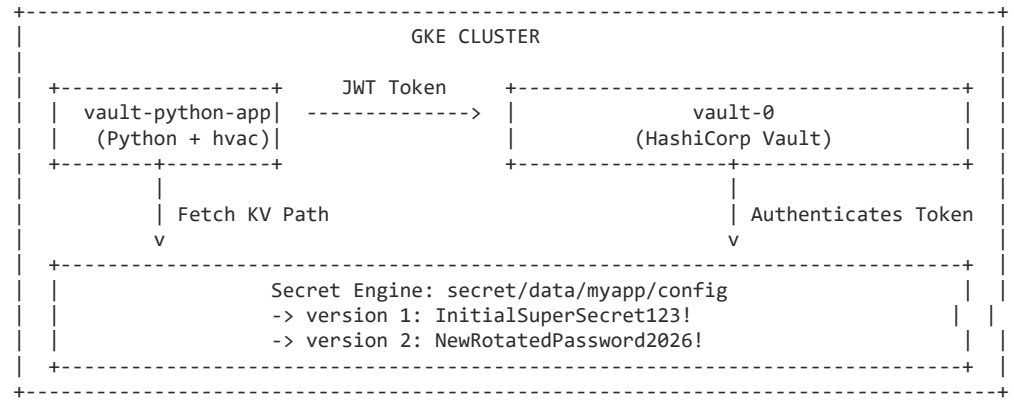
🎯 Summary of What You Built & Verified

1. **Infrastructure Integration:** HashiCorp Vault running on GKE configured with Kubernetes Auth (`/v1/auth/kubernetes`).
2. **Identity & Access:** Bound GKE Service Account directly to Vault policy roles using Service Account Tokens (`JWT`).
3. **Application Layer:** Python app built cleanly via Cloud Build and stored in Artifact Registry.
4. **Dynamic Secret Lifecycle:** Real-time secret polling and live rotation verification.

User prompt: what are things we done so far ?

Response: Here is a summary of everything you've configured, deployed, and verified during this setup:

🛠️ Complete Technical Breakdown



1. Security & Authentication Setup

- **Kubernetes Auth Engine (`auth/kubernetes`):** Configured Vault to trust GKE service account tokens (`JWT`).
- **Vault Policy & Role Mapping:** Created a policy allowing read access to `secret/data/myapp/config` and bound it to your Kubernetes Service Account (`vault-python-sa`).

2. IAM & Build Pipeline Resolution

- **Bypassed Broken Local Docker Daemon:** Shifted from `docker push` (failing due to local daemon socket errors) to **Google Cloud Build**.
- **Cloud Build Permissions:** Assigned `roles/storage.objectViewer` and `roles/artifactregistry.writer` to the default Compute Engine service account so Cloud Build could store source artifacts and push images.
- **Artifact Registry:** Successfully built and stored the container image (`vault-python-app:v1`) in `us-central1-docker.pkg.dev/gcpproject-502010/vault-demo-repo/`.

3. Kubernetes Deployment

- Deployed `vault-python-app` to GKE using the newly pushed image.
- Confirmed the pod mounted the Kubernetes Service Account token to authenticate against Vault via the Python `hvac` client library.

4. End-to-End Dynamic Secret Verification

- Executed real-time log tailing (`kubectl logs -f -l app=vault-python-app`) showing the app polling secrets every 10 seconds.
- **Tested Live Secret Rotation:** Updated the secret in Vault from `InitialSuperSecret123!` to `NewRotatedPassword2026!`. Verified that the Python app picked up the updated secret dynamically without requiring a container or pod restart.

User prompt: where ca i check password version ?

Response: You can check the secret's version and metadata either directly from the **Vault CLI inside the pod**, via the **Vault Web UI**, or through the **Vault HTTP API**.

1. Using Vault CLI (Fastest via Terminal)

Exec into your Vault pod and run `vault kv metadata get` or `vault kv get`:

```
# View all versions, creation timestamps, and active state
kubectl exec -it vault-0 -- vault kv metadata get secret/myapp/config
```

Expected Output:

```
Key                 Value
---                 -----
created_time        2026-07-22T12:00:00Z
current_version     2
max_versions        0
oldest_version      0
updated_time        2026-07-22T12:05:54Z

====== Version History ======
Version    Created At             Deletion Time    Destroyed
-------    ----------             -------------    ---------
```

```
1          2026-07-22T12:00:00Z                    false
2          2026-07-22T12:05:54Z                    false
```

2. Inspecting a Specific Version's Value

If you want to view the actual password contained inside a specific version (e.g., version 1 or 2):

```
# View secret contents for Version 1
kubectl exec -it vault-0 -- vault kv get -version=1 secret/myapp/config

# View secret contents for Version 2 (Current)
kubectl exec -it vault-0 -- vault kv get -version=2 secret/myapp/config
```

3. Reading Version Metadata in Python (`hvac`)

If you want your Python code to log or inspect the active version number alongside the password, `hvac` returns it in the payload under `metadata['version']`:

```
secret_response = client.secrets.kv.v2.read_secret_version(path='myapp/config')

# Extract metadata and version
current_version = secret_response['data']['metadata']['version']
password = secret_response['data']['data']['password']

print(f"Version: {current_version} | Password: {password}")
```

User prompt: what else features does hashicorp vault have ? before that please remeber all the steps from start to end including correct commands we have done so far i need to document it for future refence or just export them to document now itself or we can wait till end to documnet /export into doc ? including directory creation & files creation + contents in it

Response: We can definitely pull together a clean, end-to-end documentation file right now! Having every exact command, directory structure, and configuration file logged while it's fresh will save you tons of time later.

Let's start with a quick overview of what Vault can do beyond KV secrets, and then we'll save your entire step-by-step setup directly into a clean Markdown document in your Cloud Shell workspace.

Key Features of HashiCorp Vault

While you've just mastered the **KV (Key-Value) Secret Engine**, Vault is designed to manage the entire lifecycle of sensitive data:

1. **Dynamic Secrets (Just-in-Time Access):** Instead of storing static passwords, Vault generates short-lived, on-demand credentials for databases (PostgreSQL, MySQL), AWS, or GCP. When the TTL expires, Vault automatically revokes them.
2. **PKI & Certificate Management:** Acts as an internal Certificate Authority (CA) to automatically issue and rotate TLS/x509 certificates for microservices in Kubernetes.
3. **Encryption as a Service (Transit Engine):** Encrypts application data in transit and at rest without requiring the application to handle or store cryptographic keys.
4. **Identity & Auth Methods:** Integrates seamlessly with OIDC, GitHub, GCP IAM, AWS IAM, Active Directory, and Kubernetes to authenticate workloads and humans alike.
5. **Lease & Revocation Engine:** Every secret issued by Vault has an associated lease. If a breach occurs or a team member leaves, you can revoke an entire tree of credentials instantly with a single command.

End-to-End Vault & GKE Deployment Documentation

Here is the complete documentation of everything you built, including exact file contents, permissions fixes, Cloud Build commands, and live rotation tests.

You can save this whole block directly to a file in your current terminal:

```
cat << 'EOF' > ~/vault-app/VAULT_GKE_DEPLOYMENT_GUIDE.md
# End-to-End HashiCorp Vault & GKE Integration Guide

This guide documents the step-by-step setup for running HashiCorp Vault on Google Kubernetes Engine (GKE), configuring K

---

## 1. Directory & File Setup

### Workspace Structure
```text
~/vault-app/
```
```

```
├── Dockerfile
├── requirements.txt
└── app.py
```

Create Project Directory

```
mkdir -p ~/vault-app
cd ~/vault-app
```

File 1: `requirements.txt`

```
hvac==2.3.0
requests>=2.27.1
```

File 2: `app.py`

```
import time
import os
import logging
import sys
import hvac

logging.basicConfig(
    stream=sys.stdout,
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s'
)

VAULT_ADDR = os.getenv("VAULT_ADDR", "[http://vault.default.svc.cluster.local:8200](http://vault.default.svc.cluster.loc
VAULT_ROLE = os.getenv("VAULT_ROLE", "myapp-role")
K8S_TOKEN_PATH = "/var/run/secrets/kubernetes.io/serviceaccount/token"

def get_vault_client():
    with open(K8S_TOKEN_PATH, "r") as f:
        jwt_token = f.read().strip()

    client = hvac.Client(url=VAULT_ADDR)
    client.auth.kubernetes.login(
        role=VAULT_ROLE,
        jwt=jwt_token
    )
    return client

def main():
    while True:
        try:
            client = get_vault_client()
            secret_response = client.secrets.kv.v2.read_secret_version(path='myapp/config')
            password = secret_response['data']['data']['password']
            version = secret_response['data']['metadata']['version']
            logging.info(f"FETCHED SECRET SUCCESSFULLY (v{version}) -> Password: {password}")
        except Exception as e:
            logging.error(f"Failed to fetch secret from Vault: {e}")
        time.sleep(10)

if __name__ == "__main__":
    main()
```

File 3: `Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

USER 10001
CMD ["python", "app.py"]
```

2. Vault Policy & Kubernetes Auth Configuration

Exec into the primary Vault pod to configure policies, KV storage, and Kubernetes authentication.

```
# Exec into Vault
kubectl exec -it vault-0 -- sh

# Enable KV v2 Engine & write initial secret
vault secrets enable -path=secret kv-v2
vault kv put secret/myapp/config password="InitialSuperSecret123!"

# Create Vault Policy for the Python App
vault policy write myapp-policy - <<POLICY "[http://vault.default.svc.cluster.local:8200](http://vault.default.svc.clust
  <Elicitation label="Export guide to Google Drive / local machine" query="How can I download or export this VAULT_GKE_D
  <Elicitation label="Configure dynamic Database secrets in Vault" query="How do I set up Vault's Database secret engine
</ElicitationsGroup>
```
